INUKA HACKATHON -- STAGE 1: DATA ENGINEERING -- PROBLEM 7D

Closing KPC's Order-to-Cash Leakage Gap

Reconciling loading, dispatch, invoicing, and payment -- automatically, with evidence.

Lameck Irungu | 24 July 2026

THE PROBLEM

Revenue leaks at every handoff -- and nobody finds out until the audit.

- KPC's order-to-cash cycle runs through 4 handoffs: depot loading, gate dispatch, finance invoicing, AR payment -- each owned by a different system.
- Today, reconciling them is manual, periodic, or doesn't happen at all.
- Every handoff is a place product or revenue can go missing -- with zero visibility until a month-end audit tries to explain the gap.
- This isn't a data-quality nicety. It's lost cash AND a lost audit trail: KPC can't show a regulator or lender where a shilling of revenue went.

WHAT WE BUILT

An automated pipeline -- not a spreadsheet audit.

- Ingest 4 messy, independently-owned exports (mixed date formats, currency-formatted numbers, depot-name aliasing, missing fields).
- Clean + validate against explicit schemas, with a data-quality gate that fails hard on structural defects -- but never on the business gaps we exist to measure.
- Reconcile the full loading -> dispatch -> invoice -> payment chain, attributing every leaked shilling to exactly one of 5 categories.
- Ship it as a real product: a FastAPI backend + React dashboard, containerized, with CI that lints, tests, and re-runs the quality gate on every push.

THE TAXONOMY

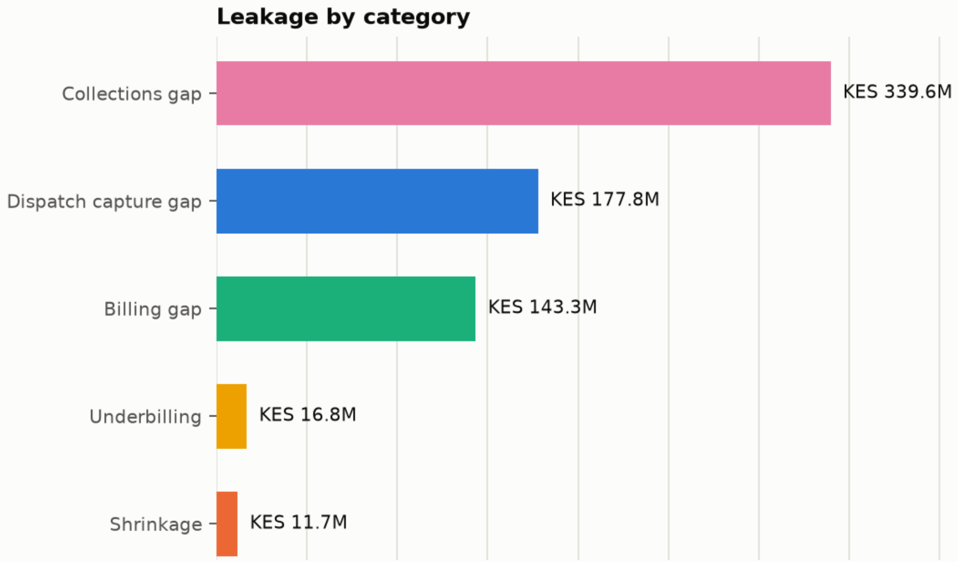
Five categories. Mutually exclusive. Zero double-counting.

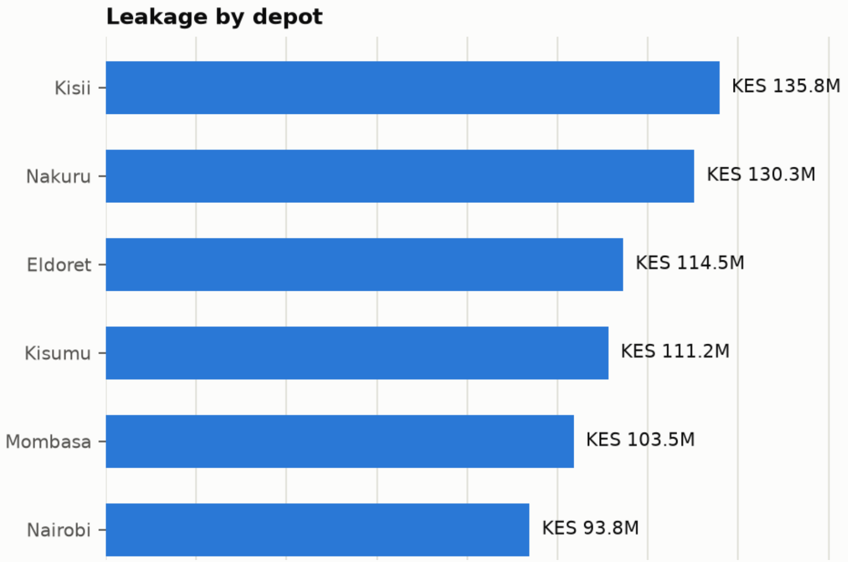
Dispatch capture gap	Loaded, but no dispatch record exists at all
Shrinkage	Dispatched volume is materially below loaded volume
Billing gap	Confirmed dispatch, but never invoiced
Underbilling	Invoiced for less volume than was dispatched
Collections gap	Invoiced correctly, but payment is partial or outstanding

EVIDENCE

5.5% of expected revenue never converts to cash.

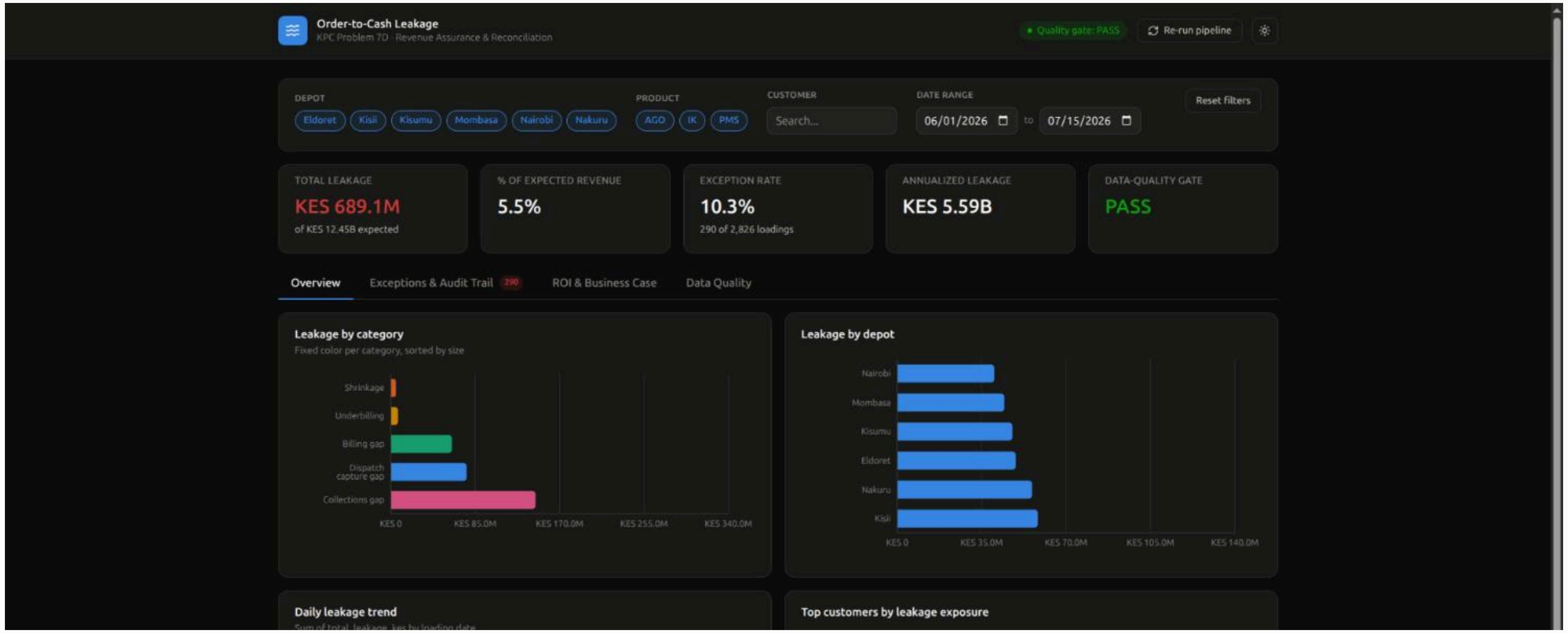
TOTAL LEAKAGE (45 DAYS)	% OF EXPECTED REVENUE	ANNUALIZED	EXCEPTIONS FLAGGED
KES 689.1M	5.5%	KES 5.59B	290 / 2,826





Synthetic dataset calibrated to plausible KPC depot volumes -- illustrative, not live data.

A real dashboard -- live filters, audit trail, ROI.



React + TypeScript + Recharts. Filters call the API live -- not a static export.

IMPACT

Targeted fixes close most of the gap.

YEAR-1 BUILD COST	YEAR-1 REALIZED BENEFIT	PAYBACK PERIOD	YEAR-1 ROI MULTIPLE
KES 75.0M	**KES 1.18B**	**0.8 months**	**15.8x**

Kisii depot and Baraka Downstream are the single largest depot- and customer-level exposures. Leakage concentrates -- a few targeted interventions, not a company-wide program, close most of the gap.

Illustrative figures, scaled from a 45-day simulation -- re-validate against live data.

WHAT'S NEXT

Stage 1 is the foundation. Stage 2 builds intelligence on it.

- Stage 2: connect to live KPC extracts; statistical diagnostics and a predictive shrinkage/collections-risk model on top of this reconciliation.
- Stage 3: harden into the monitored, CI/CD-deployed service this repo already points at; validate the ROI model against real recovery data.
- The ask: KPC operational data access to replace the synthetic dataset and turn this from a directional estimate into a validated business case.

Thank you -- questions?